

Package ‘GeneExprClust’

June 8, 2025

Type Package

Title Gene Expression Clustering and Enrichment Analysis

Version 0.1.0

Description Provides functions for reading, normalizing, clustering gene expression data, performing gene set enrichment analysis (GSEA), and visualization. Supports multiple clustering algorithms (hierarchical, k-means, DBSCAN), normalization methods (log2, quantile, z-score, DESeq2), and integrates with clusterProfiler for enrichment.

License MIT + file LICENSE

Encoding UTF-8

Imports BiocParallel,
ggdendro,
Biobase,
dbscan,
ggplot2,
preprocessCore,
DESeq2,
sva,
reshape2,
ComplexHeatmap,
matrixStats,
stats,
proxy,
cluster,
msigdbr,
scales,
stringr

Suggests testthat (>= 3.0.0),
knitr,
rmarkdown,
mockery,
ALL,
dplyr,
mclust,
org.Hs.eg.db

RoxygenNote 7.3.2

Config/testthat/edition 3

VignetteBuilder knitr

biocViews Clustering, GeneExpression, Transcriptomics

URL notavailable

BugReports notavailable

Depends R (>= 4.4.0)

Contents

cluster_expression	2
create_example_gene_sets	3
dbscan_cluster	4
detect_outliers	5
eset	6
extract_gene_names	6
filter_low_expression	7
gap_statistic	8
gap_statistic_parallel	9
get_cluster_results	10
hierarchical_cluster	11
kmeans_cluster	12
kmeans_parallel_k	13
load_gmt_gene_sets	14
load_online_gene_sets	14
normalize_data	15
ora_from_expression_set	16
perform_batch_correction	17
perform_normalization	17
perform_ora	18
perform_ora_parallel	19
plot_cluster_profiles	20
plot_dbscan_results	20
plot_dendrogram	21
plot_elbow	22
plot_heatmap	22
plot_pca	23
read_expression	24
run_pipeline	25
silhouette_score	28
summarize_ora_results	29
wss_score	30

cluster_expression	<i>Unified clustering function with multiple methods</i>
--------------------	--

Description

Unified clustering function with multiple methods

Usage

```
cluster_expression(  
  eset,  
  method = c("hierarchical", "kmeans", "dbscan"),  
  k = 3,  
  target = "gene",  
  ...  
)
```

Arguments

eset	ExpressionSet
method	One of "hierarchical", "kmeans", "dbscan"
k	Number of clusters (used for hierarchical and kmeans)
target	Cluster by "gene" (default) or "sample"
...	Additional parameters passed to method

Details

If ‘target = "sample"’, samples (columns) are clustered by transposing the expression matrix. If ‘target = "gene"’, gene (rows) are clustered directly. The ‘cluster’ vector in the returned object is named by gene or sample names accordingly.

Value

A list with model object and cluster assignments

Examples

```
# Hierarchical clustering with automatic cluster assignment  
data(eset)  
hc_result <- cluster_expression(eset, method = "hierarchical", k = 4)  
table(hc_result$clusters)  
  
# K-means clustering  
km_result <- cluster_expression(eset, method = "kmeans", k = 3, target = "sample")  
print(km_result$clusters)  
  
# DBSCAN clustering  
db_result <- cluster_expression(eset, method = "dbscan", eps = 2.0, minPts = 5)  
summary(as.factor(db_result$clusters))
```

`create_example_gene_sets`*Create Simple Example Gene Sets*

Description

Creates example gene sets for testing purposes.

Usage

```
create_example_gene_sets(gene_universe, n_pathways = 50, pathway_size = 30)
```

Arguments

<code>gene_universe</code>	Vector of all available gene names
<code>n_pathways</code>	Number of pathways to create
<code>pathway_size</code>	Average size of each pathway

Value

Named list of example gene sets

Examples

```
data(eset)
# Create example gene sets
example_sets <- create_example_gene_sets(rownames(eset), n_pathways = 20)
```

`dbscan_cluster`*Cluster genes using DBSCAN*

Description

Cluster genes using DBSCAN

Usage

```
dbscan_cluster(
  eset,
  eps,
  minPts = 5,
  distance = "euclidean",
  scale = TRUE,
  target = "gene"
)
```

Arguments

eset	ExpressionSet object
eps	Numeric, DBSCAN epsilon parameter (required)
minPts	Integer, minimum number of points per cluster (default 5)
distance	Character, distance metric ("euclidean" or "correlation")
scale	Logical, whether to z-score normalize expression values (default TRUE)
target	Cluster "gene" (rows) or "sample" (columns)

Value

List containing:

- cluster - Named vector of cluster assignments (0 = noise)
- eps - The eps parameter used
- minPts - The minPts parameter used

Examples

```
# DBSCAN clustering of genes
data(eset)
db_genes <- dbscan_cluster(eset, eps = 2.5, minPts = 3, target = "gene")
table(db_genes$cluster) # 0 indicates noise points

# DBSCAN clustering of samples with correlation distance
db_samples <- dbscan_cluster(eset, eps = 0.3, minPts = 2,
                             distance = "correlation", target = "sample")
print(db_samples$cluster)

# Check for noise points (cluster = 0)
noise_genes <- names(db_genes$cluster)[db_genes$cluster == 0]
message(paste("Number of noise genes: ", length(noise_genes)))
```

detect_outliers

Detect and optionally remove outlier genes based on variability

Description

Detect and optionally remove outlier genes based on variability

Usage

```
detect_outliers(
  eset,
  method = c("mad", "iqr"),
  threshold = 3,
  plot = FALSE,
  remove = TRUE
)
```

Arguments

eset	An ExpressionSet object
method	Method for variability detection: "mad" (default) or "iqr"
threshold	Threshold multiplier (e.g., 3 MADs above median)
plot	Logical. If TRUE, shows a distribution plot of variability
remove	Logical. If TRUE (default), removes outlier genes from the returned ExpressionSet.

Value

A list with:

filtered	ExpressionSet (original or filtered depending on 'remove')
outliers	Character vector of outlier gene names

Examples

```
data(eset)
# Detect and remove outlier genes using MAD method
eset_filtered <- detect_outliers(eset, plot = TRUE)
```

eset	<i>Example Expression Set</i>
------	-------------------------------

Description

A toy dataset for demonstrating clustering.

Usage

```
data(eset)
```

Format

A matrix with 10 rows and 10 columns.

Examples

```
data(eset)
str(eset)
```

extract_gene_names	<i>Extract Gene Names from ExpressionSet</i>
--------------------	--

Description

Helper function to extract gene names from Bioconductor ExpressionSet objects.

Usage

```
extract_gene_names(eset, id_type = "featureNames")
```

Arguments

eset	ExpressionSet object
id_type	Type of gene identifiers to extract ("featureNames", "SYMBOL", "ENTREZID", etc.)

Value

Character vector of gene names

Examples

```
# Create a sample ExpressionSet
library(Biobase)

# Create expression matrix
expr_matrix <- matrix(rnorm(100), nrow = 10, ncol = 10)
rownames(expr_matrix) <- paste0("probe_", 1:10)
colnames(expr_matrix) <- paste0("sample_", 1:10)

# Create feature data with gene symbols
feature_data <- data.frame(
  SYMBOL = c("GENE1", "GENE2", "GENE3", "GENE4", "GENE5",
             "GENE6", "GENE7", "GENE8", "GENE9", "GENE10"),
  ENTREZID = paste0("ID_", 1:10),
  stringsAsFactors = FALSE
)
rownames(feature_data) <- rownames(expr_matrix)

# Create phenotype data
pheno_data <- data.frame(
  condition = rep(c("control", "treatment"), each = 5),
  stringsAsFactors = FALSE
)
rownames(pheno_data) <- colnames(expr_matrix)

# Create ExpressionSet
eset <- ExpressionSet(
  assayData = expr_matrix,
  phenoData = AnnotatedDataFrame(pheno_data),
  featureData = AnnotatedDataFrame(feature_data)
)
```

```
# Extract gene names using different methods
probe_ids <- extract_gene_names(eset) # Default: featureNames
gene_symbols <- extract_gene_names(eset, id_type = "SYMBOL")
entrez_ids <- extract_gene_names(eset, id_type = "ENTREZID")
```

`filter_low_expression` *Filter low-expressed genes from expression matrix and ExpressionSet*

Description

Filter low-expressed genes from expression matrix and ExpressionSet

Usage

```
filter_low_expression(expr, eset, filter_threshold = 1, filter_min_samples = 3)
```

Arguments

`expr` Numeric expression matrix (genes x samples).
`eset` ExpressionSet object.
`filter_threshold` Numeric, minimum expression threshold to consider a gene expressed.
`filter_min_samples` Integer, minimum number of samples that must meet the threshold.

Value

List with filtered expression matrix ('expr') and ExpressionSet ('eset').

`gap_statistic` *Compute Gap Statistic using k-means*

Description

Compute Gap Statistic using k-means

Usage

```
gap_statistic(data, max_k = 10, B = 20)
```

Arguments

`data` Numeric matrix or data frame (genes x samples)
`max_k` Maximum number of clusters to try (default 10)
`B` Number of bootstrap samples (default 20)

Value

Data frame with gap statistic per K

Examples

```
# Load required package
library(cluster)

# Create sample data with natural clustering structure
set.seed(789)
# Three natural clusters
cluster1 <- matrix(rnorm(60, mean = 0, sd = 0.5), nrow = 15, ncol = 4)
cluster2 <- matrix(rnorm(60, mean = 2, sd = 0.5), nrow = 15, ncol = 4)
cluster3 <- matrix(rnorm(60, mean = -1.5, sd = 0.5), nrow = 15, ncol = 4)

data <- rbind(cluster1, cluster2, cluster3)
rownames(data) <- paste0("Gene", 1:45)
colnames(data) <- paste0("Sample", 1:4)

# Compute gap statistic
gap_result <- gap_statistic(data, max_k = 8, B = 10)

# View results
print(gap_result)

# Find optimal k (first local maximum)
optimal_k <- which.max(gap_result$gap)
message(paste("Suggested optimal k: ", optimal_k))

# Plot gap statistic
plot(1:nrow(gap_result), gap_result$gap, type = "b",
     xlab = "Number of clusters (k)", ylab = "Gap statistic",
     main = "Gap Statistic for Optimal k")
abline(v = optimal_k, col = "red", lty = 2)
```

gap_statistic_parallel

Parallel Gap Statistic Computation

Description

Parallel Gap Statistic Computation

Usage

```
gap_statistic_parallel(
  data,
  max_k = 10,
  B = 20,
  BPPARAM = BiocParallel::bpparam()
)
```

Arguments

data	Numeric matrix or data frame (genes x samples)
max_k	Maximum number of clusters to try (default 10)
B	Number of bootstrap samples (default 20)
BPPARAM	BiocParallel backend (default: bpparam())

Value

Data frame with gap statistic per K

Examples

```
# Load required packages
library(cluster)
library(BiocParallel)

# Create larger sample dataset for parallel processing demonstration
set.seed(999)
# Create 4 natural clusters with more samples
n_genes <- 200
n_samples <- 20

cluster1 <- matrix(rnorm(n_genes/4 * n_samples, mean = 0),
  nrow = n_genes/4, ncol = n_samples)
cluster2 <- matrix(rnorm(n_genes/4 * n_samples, mean = 2),
  nrow = n_genes/4, ncol = n_samples)
cluster3 <- matrix(rnorm(n_genes/4 * n_samples, mean = -2),
  nrow = n_genes/4, ncol = n_samples)
cluster4 <- matrix(rnorm(n_genes/4 * n_samples, mean = 1.5),
  nrow = n_genes/4, ncol = n_samples)

large_data <- rbind(cluster1, cluster2, cluster3, cluster4)
rownames(large_data) <- paste0("Gene", 1:n_genes)
colnames(large_data) <- paste0("Sample", 1:n_samples)

# Set up parallel backend (adjust workers based on your system)
param <- MulticoreParam(workers = 2) # Use 2 cores

# Compute gap statistic in parallel
system.time({
  gap_parallel_result <- gap_statistic_parallel(
    large_data,
    max_k = 8,
    B = 50, # More bootstrap samples
    BPPARAM = param
  )
})

# View results
print(gap_parallel_result)

# Compare with sequential version timing
system.time({
  gap_sequential_result <- gap_statistic(large_data, max_k = 8, B = 50)
})

# Plot comparison
plot(1:nrow(gap_parallel_result), gap_parallel_result$gap, type = "b",
  xlab = "Number of clusters (k)", ylab = "Gap statistic",
  main = "Parallel Gap Statistic Results", col = "blue")
```

get_cluster_results	<i>Get Detailed Results for Specific Cluster</i>
---------------------	--

Description

Extract detailed ORA results for a specific cluster.

Usage

```
get_cluster_results(  
  ora_results,  
  cluster_id,  
  p_threshold = 0.05,  
  include_genes = TRUE  
)
```

Arguments

ora_results	Output from perform_ora()
cluster_id	Cluster ID to extract results for
p_threshold	P-value threshold for filtering
include_genes	Whether to include the list of overlapping genes

Value

Data frame of detailed results for the specified cluster

Examples

```
## Not run:  
# Get results for specific cluster  
cluster1_results <- get_cluster_results(ora_results, cluster_id = 1)  
  
## End(Not run)
```

hierarchical_cluster	<i>Hierarchical clustering of genes or samples</i>
----------------------	--

Description

Hierarchical clustering of genes or samples

Usage

```
hierarchical_cluster(  
  eset,  
  hclust_method = "ward.D2",  
  distance = "euclidean",  
  target = "gene"  
)
```

Arguments

<code>eset</code>	ExpressionSet
<code>hclust_method</code>	Linkage method (default: "ward.D2")
<code>distance</code>	Distance metric (default: "euclidean")
<code>target</code>	Cluster by "gene" (default) or "sample"

Details

If `target = "sample"`, samples (columns) are clustered by transposing the expression matrix. If `target = "gene"`, gene (rows) are clustered directly.

Value

`hclust` object

Examples

```
# Load required libraries
library(Biobase)

# Create sample ExpressionSet
set.seed(123)
expr_matrix <- matrix(rnorm(1000), nrow = 100, ncol = 10)
rownames(expr_matrix) <- paste0("Gene_", 1:100)
colnames(expr_matrix) <- paste0("Sample_", 1:10)
eset <- ExpressionSet(assayData = expr_matrix)

# Hierarchical clustering of genes
hc_genes <- hierarchical_cluster(eset, target = "gene")
plot(hc_genes, main = "Gene Clustering")

# Hierarchical clustering of samples
hc_samples <- hierarchical_cluster(eset, target = "sample",
                                  hclust_method = "complete")
plot(hc_samples, main = "Sample Clustering")
```

kmeans_cluster

K-means clustering of genes or samples

Description

K-means clustering of genes or samples

Usage

```
kmeans_cluster(eset, k = 5, nstart = 25, target = "gene", ...)
```

Arguments

eset	ExpressionSet
k	Number of clusters
nstart	Number of random starts (default: 25)
target	Cluster by "gene" (default) or "sample"
...	Other params used by other methods

Details

If ‘target = "sample"’, samples (columns) are clustered by transposing the expression matrix. If ‘target = "gene"’, gene (rows) are clustered directly.

Value

kmeans object

Examples

```
# K-means clustering of genes into 5 clusters
data(eset)
km_genes <- kmeans_cluster(eset, k = 5, target = "gene")
table(km_genes$cluster)
```

kmeans_parallel_k	<i>Parallel K-means with Multiple K Values</i>
-------------------	--

Description

Parallel K-means with Multiple K Values

Usage

```
kmeans_parallel_k(
  eset,
  k_range = 2:10,
  target = "gene",
  BPPARAM = BiocParallel::bpparam()
)
```

Arguments

eset	ExpressionSet
k_range	Vector of k values to try
target	Cluster by "gene" (default) or "sample"
BPPARAM	BiocParallel backend

Value

List of kmeans results for each k

Examples

```
data(eset)
# Parallel k-means for multiple k values on genes
library(BiocParallel)
results <- kmeans_parallel_k(eset, k_range = 2:8, target = "gene")

# Extract results for k=5
k5_result <- results$k_5
table(k5_result$cluster)

# Compare total within-cluster sum of squares across k values
sapply(results, function(x) x$tot.withinss)
```

load_gmt_gene_sets	<i>Load Gene Sets from GMT File</i>
--------------------	-------------------------------------

Description

Helper function to load gene sets from GMT (Gene Matrix Transposed) format files. This is a common format for pathway databases like MSigDB.

Usage

```
load_gmt_gene_sets(gmt_file, min_size = 5, max_size = 500)
```

Arguments

gmt_file	Path to GMT file
min_size	Minimum size of gene sets to keep
max_size	Maximum size of gene sets to keep

Value

Named list of gene

Examples

```
## Not run:
# Load gene sets from GMT file
gene_sets <- load_gmt_gene_sets("pathways.gmt")

## End(Not run)
```

load_online_gene_sets *Load Gene Sets from Online Databases*

Description

Fetch gene sets directly from online databases using msigdb package. This connects to servers automatically without needing file downloads.

Usage

```
load_online_gene_sets(
  species = "Homo sapiens",
  collection = "H",
  subcategory = NULL,
  min_size = 5,
  max_size = 500
)
```

Arguments

species	Species name (e.g., "Homo sapiens", "Mus musculus")
collection	MSigDB category (e.g., "H" for Hallmark, "C2" for curated, "C5" for GO)
subcategory	Optional subcategory (e.g., "GO:BP" for GO Biological Process)
min_size	Minimum size of gene sets to keep
max_size	Maximum size of gene sets to keep

Value

Named list of gene sets

Examples

```
# Get Hallmark pathways for human
hallmark_sets <- load_online_gene_sets("Homo sapiens", "H")
# Load Hallmark pathways for human
hallmark_sets <- load_online_gene_sets("Homo sapiens", "H")
```

normalize_data *Normalize expression data in an ExpressionSet with optional batch correction and filtering*

Description

Supports log2 transformation, quantile normalization, z-score standardization, DESeq2 median-of-ratios normalization, optional batch correction (ComBat), and low-expression filtering.

Usage

```
normalize_data(
  eset,
  method = c("log2", "quantile", "zscore", "deseq"),
  batch = NULL,
  filter = FALSE,
  filter_threshold = 10,
  filter_min_samples = 3
)
```

Arguments

<code>eset</code>	ExpressionSet object to normalize.
<code>method</code>	Normalization method: "log2", "quantile", "zscore", or "deseq".
<code>batch</code>	Optional batch factor for ComBat batch correction (default NULL).
<code>filter</code>	Logical, whether to filter low-expressed genes (default FALSE).
<code>filter_threshold</code>	Numeric, minimum count threshold for filtering (default 10).
<code>filter_min_samples</code>	Integer, minimum number of samples meeting threshold (default 3).

Value

Normalized ExpressionSet.

Examples

```
data(eset)
# Log2 normalization
eset_log2 <- normalize_data(eset, method = "log2")
```

ora_from_expression_set

Perform ORA Analysis from ExpressionSet and Clusters

Description

Convenience wrapper function that takes an ExpressionSet and cluster results to perform ORA analysis.

Usage

```
ora_from_expression_set(
  eset,
  clusters,
  gene_sets,
  gene_id_type = "featureNames",
  ...
)
```


Arguments

eset	ExpressionSet object containing gene expression data
clusters	Named vector of cluster assignments
gene_sets	Named list of gene sets for enrichment testing
gene_id_type	Type of gene IDs to use from ExpressionSet
...	Additional arguments passed to perform_ora()

Value

List of ORA results per cluster

Examples

```
## Not run:  
# ORA analysis from ExpressionSet  
ora_results <- ora_from_expression_set(eset, clusters, gene_sets)  
  
## End(Not run)
```

perform_batch_correction

Perform batch correction on expression matrix using ComBat

Description

Perform batch correction on expression matrix using ComBat

Usage

```
perform_batch_correction(expr, batch)
```

Arguments

expr	Numeric matrix of expression values (genes x samples).
batch	Factor or vector indicating batch assignment per sample.

Value

Batch-corrected expression matrix.

perform_normalization *Perform normalization on an expression matrix*

Description

Supports "log2", "quantile", "zscore", and "deseq" normalization methods.

Usage

```
perform_normalization(expr, method)
```

Arguments

expr	Numeric matrix of expression values (genes x samples).
method	Normalization method, one of "log2", "quantile", "zscore", or "deseq".

Value

Normalized expression matrix.

perform_ora *Perform Over-Representation Analysis (ORA) for Gene Clusters*

Description

Performs hypergeometric test for pathway enrichment in gene clusters. This is the main enrichment analysis function for clustered gene expression data.

Usage

```
perform_ora(
  clusters,
  gene_sets,
  universe = NULL,
  p_adjust_method = "BH",
  min_set_size = 5,
  max_set_size = 500,
  p_threshold = 0.05
)
```

Arguments

clusters	Named vector of cluster assignments (gene names as names, cluster IDs as values)
gene_sets	Named list of gene sets/pathways (each element is a vector of gene names)
universe	Vector of background genes. If NULL, uses all genes from clusters and gene_sets
p_adjust_method	Method to adjust p-values for multiple testing (default: "BH")
min_set_size	Minimum number of genes required in a gene set to be tested
max_set_size	Maximum number of genes allowed in a gene set to be tested
p_threshold	P-value threshold for significance (used in summary functions)

Value

List of ORA results per cluster, each containing a data frame with enrichment statistics

Examples

```
# Basic ORA analysis
clusters <- c(1, 1, 2, 2, 3, 3, 1, 2)
names(clusters) <- c("GENE1", "GENE2", "GENE3", "GENE4", "GENE5",
                    "GENE6", "GENE7", "GENE8")

gene_sets <- list(
  Pathway1 = c("GENE1", "GENE2", "GENE7", "GENE9", "GENE10"),
  Pathway2 = c("GENE3", "GENE4", "GENE8", "GENE11", "GENE12")
)
ora_results <- perform_ora(clusters, gene_sets)
```

perform_ora_parallel *Parallel Over-Representation Analysis (ORA) for Gene Clusters*

Description

Parallel Over-Representation Analysis (ORA) for Gene Clusters

Usage

```
perform_ora_parallel(
  clusters,
  gene_sets,
  universe = NULL,
  p_adjust_method = "BH",
  min_set_size = 5,
  max_set_size = 500,
  p_threshold = 0.05,
  BPPARAM = BiocParallel::bpparam()
)
```

Arguments

clusters	Named vector of cluster assignments
gene_sets	Named list of gene sets/pathways
universe	Vector of background genes
p_adjust_method	Method to adjust p-values (default: "BH")
min_set_size	Minimum gene set size
max_set_size	Maximum gene set size
p_threshold	P-value threshold for significance
BPPARAM	BiocParallel backend

Value

List of ORA results per cluster

Examples

```
## Not run:
# Parallel ORA analysis
ora_results <- perform_ora_parallel(clusters, gene_sets)

## End(Not run)
```

`plot_cluster_profiles` *Plot average gene expression profiles per cluster*

Description

Plot average gene expression profiles per cluster

Usage

```
plot_cluster_profiles(eset, clusters, scale = FALSE)
```

Arguments

<code>eset</code>	ExpressionSet object
<code>clusters</code>	Named vector of cluster assignments for genes
<code>scale</code>	Logical, whether to z-score normalize gene expression before averaging (default FALSE)

Value

ggplot object showing average expression profile per cluster across samples

Examples

```
expr_matrix <- matrix(rnorm(1000), nrow = 100, ncol = 10)
colnames(expr_matrix) <- paste0("Sample", 1:10)
rownames(expr_matrix) <- paste0("Gene", 1:100)
eset <- Biobase::ExpressionSet(assayData = expr_matrix)
clusters <- kmeans(expr_matrix, centers = 3)$cluster
plot_cluster_profiles(eset, clusters, scale = TRUE)
```

`plot_dbscan_results` *Plot DBSCAN clustering results using PCA*

Description

Visualizes DBSCAN clustering results on genes or samples by performing PCA for dimensionality reduction and plotting the first two principal components. Noise points (cluster 0) are colored differently to distinguish from clusters.

Usage

```
plot_dbscan_results(eset, clusters, target = "gene")
```

Arguments

eset	ExpressionSet object containing expression data
clusters	Named vector of cluster assignments from DBSCAN (0 = noise)
target	Character, either "gene" (default) or "sample" to indicate whether clustering was done on genes or samples

Value

A ggplot2 object showing the PCA plot colored by cluster assignment

Examples

```
data(eset)
# Plot DBSCAN clustering results for genes
cluster_results <- c(1, 1, 2, 0, 2, 3) # Example cluster assignments
names(cluster_results) <- rownames(eset)[1:6]
plot_dbscan_results(eset, cluster_results, target = "gene")
```

plot_dendrogram	<i>Plot dendrogram from hierarchical clustering model</i>
-----------------	---

Description

Plot dendrogram from hierarchical clustering model

Usage

```
plot_dendrogram(hclust_model, main = "Hierarchical Clustering Dendrogram")
```

Arguments

hclust_model	Object returned by hclust()
main	Title for the plot (optional)

Value

ggplot object

Examples

```
expr_matrix <- matrix(rnorm(1000), nrow = 100, ncol = 10)
colnames(expr_matrix) <- paste0("Sample", 1:10)
rownames(expr_matrix) <- paste0("Gene", 1:100)
eset <- Biobase::ExpressionSet(assayData = expr_matrix)
hc <- hclust(dist(Biobase::exprs(eset)))
p <- plot_dendrogram(hc)
message(p)
```

plot_elbow	<i>Elbow plot for selecting number of clusters in k-means</i>
------------	---

Description

Elbow plot for selecting number of clusters in k-means

Usage

```
plot_elbow(eset, max_k = 10, target = "gene")
```

Arguments

eset	ExpressionSet
max_k	Maximum number of clusters to try
target	Cluster by "gene" (default) or "sample"

Details

Computes total within-cluster sum of squares (WSS) for $k = 1$ to max_k . Plots WSS to help visually select the optimal number of clusters using the elbow method. If `target = "gene"`, genes are clustered by transposing the expression matrix; otherwise samples are clustered directly.

Value

A ggplot object

Examples

```
data(eset)
# Generate elbow plot for gene clustering
elbow_plot <- plot_elbow(eset, max_k = 9, target = "gene")
message(elbow_plot)

# Generate elbow plot for sample clustering
elbow_plot_samples <- plot_elbow(eset, max_k = 8, target = "sample")
message(elbow_plot_samples)

# The "elbow" in the plot suggests optimal number of clusters
```

plot_heatmap	<i>Plot heatmap of expression data with optional clustering annotation and scaling</i>
--------------	--

Description

Plot heatmap of expression data with optional clustering annotation and scaling

Usage

```
plot_heatmap(
  eset,
  clusters = NULL,
  main = NULL,
  show_row_names = FALSE,
  scale = c("none", "row", "column")
)
```

Arguments

eset	ExpressionSet object
clusters	Named vector of cluster assignments for genes (optional)
main	The graph's title
show_row_names	Logical, whether to display gene names (default FALSE)
scale	Character, one of "none", "row", or "column" for scaling expression data (default "none")

Value

Heatmap object

Examples

```
expr_matrix <- matrix(rnorm(1000), nrow = 100, ncol = 10)
colnames(expr_matrix) <- paste0("Sample", 1:10)
rownames(expr_matrix) <- paste0("Gene", 1:100)
gene_clusters <- kmeans(expr_matrix, centers = 3)$cluster
eset <- Biobase::ExpressionSet(
  assayData = expr_matrix
)
plot_heatmap(eset, clusters = gene_clusters, scale = "row")
```

plot_pca

PCA plot of samples

Description

PCA plot of samples

Usage

```
plot_pca(eset, color_by = NULL, main = "PCA Plot", ellipse = FALSE)
```

Arguments

eset	ExpressionSet object
color_by	Character, column name in pData(eset) to color samples by
main	Character, main title for the plot (optional)
ellipse	Logical, whether to draw group confidence ellipses (default TRUE)

Value

ggplot object

Examples

```
# Create example ExpressionSet
library(Biobase)

# Expression data (genes x samples)
expr_data <- matrix(rnorm(1000), nrow = 100, ncol = 10)
rownames(expr_data) <- paste0("Gene", 1:100)
colnames(expr_data) <- paste0("Sample", 1:10)

# Phenotype data
pheno_data <- data.frame(
  sample_id = paste0("Sample", 1:10),
  treatment = rep(c("Control", "Treated"), each = 5),
  batch = rep(c("A", "B"), times = 5)
)
rownames(pheno_data) <- colnames(expr_data)

# Create ExpressionSet
eset <- ExpressionSet(
  assayData = expr_data,
  phenoData = AnnotatedDataFrame(pheno_data)
)

# Basic PCA plot
plot_pca(eset)

# PCA plot colored by treatment group with ellipses
plot_pca(eset, color_by = "treatment", ellipse = TRUE)

# PCA plot colored by batch
plot_pca(eset, color_by = "batch", main = "PCA by Batch")
```

read_expression	<i>Read gene expression data from file</i>
-----------------	--

Description

Supports CSV, TSV, or ExpressionSet from .RData file.

Usage

```
read_expression(path, type = c("csv", "tsv", "eset"), pheno_data = NULL)
```

Arguments

path	Path to data file (.csv, .tsv, or .RData)
type	File type: "csv", "tsv", or "eset"
pheno_data	Optional data frame or file path to sample metadata

Value

ExpressionSet object

Examples

```
# Load existing ExpressionSet from RData file
file_path <- system.file("data", "eset.rda", package = "GeneExprClust")
eset <- read_expression(file_path, type = "eset")

# Check the resulting ExpressionSet
print(eset)
dim(Biobase::exprs(eset)) # Get expression matrix dimensions
Biobase::pData(eset)      # Get phenotype data
```

run_pipeline	<i>Complete Gene Expression Analysis Pipeline</i>
--------------	---

Description

This function provides a comprehensive workflow for gene expression analysis, including data loading, preprocessing, normalization, exploratory analysis, clustering, visualization, and pathway enrichment analysis.

Usage

```
run_pipeline(
  data_path = NULL,
  expression_set = NULL,
  data_type = c("csv", "tsv", "eset"),
  pheno_data = NULL,
  output_dir = "pipeline_results",
  normalize = TRUE,
  norm_method = c("log2", "quantile", "zscore", "deseq"),
  batch_correction = NULL,
  filter_low_expr = TRUE,
  filter_threshold = 1,
  filter_min_samples = 3,
  detect_outliers = TRUE,
  outlier_method = c("mad", "iqr"),
  outlier_threshold = 3,
  remove_outliers = TRUE,
  clustering_methods = c("hierarchical", "kmeans"),
  target = c("gene", "sample"),
  k_clusters = 5,
  max_k = 15,
  hclust_method = "ward.D2",
  distance = "euclidean",
  kmeans_nstart = 25,
  dbscan_eps = NULL,
  dbscan_minPts = 5,
  perform_validation = TRUE,
```

```

gap_statistic = TRUE,
gap_max_k = 10,
gap_B = 20,
create_plots = TRUE,
heatmap_scale = c("row", "none", "column"),
pca_color_by = NULL,
save_plots = TRUE,
perform_enrichment = TRUE,
gene_sets_source = c("online", "gmt", "example", "custom"),
gene_sets_collection = "H",
gene_sets_species = "Homo sapiens",
gene_sets_file = NULL,
custom_gene_sets = NULL,
enrichment_p_threshold = 0.05,
min_gene_set_size = 5,
max_gene_set_size = 500,
verbose = TRUE
)

```

Arguments

<code>data_path</code>	Character, path to expression data file or ExpressionSet .RData file
<code>expression_set</code>	The ExpressionSet object to be analyzed
<code>data_type</code>	Character, type of input data: "csv", "tsv", or "eset"
<code>pheno_data</code>	Optional data frame or file path to sample metadata
<code>output_dir</code>	Character, directory to save results and plots (default: "pipeline_results")
	# Normalization parameters
<code>normalize</code>	Logical, whether to perform normalization (default: TRUE)
<code>norm_method</code>	Character, normalization method: "log2", "quantile", "zscore", "deseq"
<code>batch_correction</code>	Character or vector, batch variable for correction (default: NULL)
<code>filter_low_expr</code>	Logical, whether to filter low-expressed genes (default: TRUE)
<code>filter_threshold</code>	Numeric, minimum expression threshold (default: 1)
<code>filter_min_samples</code>	Integer, minimum samples meeting threshold (default: 3)
	# Exploratory analysis parameters
<code>detect_outliers</code>	Logical, whether to detect and remove outlier genes (default: TRUE)
<code>outlier_method</code>	Character, method for outlier detection: "mad" or "iqr"
<code>outlier_threshold</code>	Numeric, threshold multiplier for outlier detection (default: 3)
<code>remove_outliers</code>	Logical, whether to remove detected outliers (default: TRUE)
	# Clustering parameters
<code>clustering_methods</code>	Character vector, methods to use: "hierarchical", "kmeans", "dbscan"

target	Character, cluster "gene" (default) or "sample"
k_clusters	Integer, number of clusters for hierarchical/kmeans (default: 5)
max_k	Integer, maximum k for elbow plot (default: 15)
hclust_method	Character, linkage method for hierarchical clustering (default: "ward.D2")
distance	Character, distance metric (default: "euclidean")
kmeans_nstart	Integer, number of random starts for k-means (default: 25)
dbscan_eps	Numeric, DBSCAN epsilon parameter (required if using DBSCAN)
dbscan_minPts	Integer, DBSCAN minimum points parameter (default: 5)
	# Validation parameters
perform_validation	Logical, whether to perform clustering validation (default: TRUE)
gap_statistic	Logical, whether to compute gap statistic (default: TRUE)
gap_max_k	Integer, maximum k for gap statistic (default: 10)
gap_B	Integer, bootstrap samples for gap statistic (default: 20)
	# Visualization parameters
create_plots	Logical, whether to create visualization plots (default: TRUE)
heatmap_scale	Character, scaling for heatmap: "none", "row", "column" (default: "row")
pca_color_by	Character, phenotype variable to color PCA by (default: NULL)
save_plots	Logical, whether to save plots to files (default: TRUE)
	# Enrichment analysis parameters
perform_enrichment	Logical, whether to perform pathway enrichment (default: TRUE)
gene_sets_source	Character, source for gene sets: "online", "gmt", "example", or "custom"
gene_sets_collection	Character, MSigDB collection for online source (default: "H")
gene_sets_species	Character, species for online gene sets (default: "Homo sapiens")
gene_sets_file	Character, path to GMT file if using "gmt" source
custom_gene_sets	List, custom gene sets if using "custom" source
enrichment_p_threshold	Numeric, p-value threshold for enrichment (default: 0.05)
min_gene_set_size	Integer, minimum gene set size (default: 5)
max_gene_set_size	Integer, maximum gene set size (default: 500)
	# General parameters
verbose	Logical, whether to print progress messages (default: TRUE)

Value

A list containing all analysis results:

- original_data - Original ExpressionSet
- processed_data - Processed ExpressionSet after normalization/filtering
- outlier_info - Information about detected outliers
- clustering_results - Results from all clustering methods
- validation_results - Clustering validation metrics
- plots - List of generated plots
- enrichment_results - Pathway enrichment analysis results
- summary - Summary of the analysis

Examples

```
data(eset)
results <- run_pipeline(
  expression_set = eset,
  clustering_methods = c("hierarchical", "kmeans"),
  k_clusters = 4
)
```

silhouette_score	<i>Compute silhouette width for clustering</i>
------------------	--

Description

Compute silhouette width for clustering

Usage

```
silhouette_score(data, clusters, distance = "euclidean")
```

Arguments

data	Numeric matrix or data frame (genes x samples)
clusters	Named vector of cluster assignments
distance	Character, distance metric (default = "euclidean")

Value

Silhouette object

Examples

```
# Load required packages
library(cluster)
library(proxy)

# Create sample data with clear clusters
set.seed(456)
data <- rbind(
  matrix(rnorm(50, mean = 0), nrow = 10, ncol = 5),
  matrix(rnorm(50, mean = 3), nrow = 10, ncol = 5),
  matrix(rnorm(50, mean = -2), nrow = 10, ncol = 5)
)
rownames(data) <- paste0("Gene", 1:30)

# Create cluster assignments
clusters <- c(rep(1, 10), rep(2, 10), rep(3, 10))
names(clusters) <- rownames(data)

# Compute silhouette scores
sil_result <- silhouette_score(data, clusters)

# View summary
summary(sil_result)

# Plot silhouette
plot(sil_result)
```

summarize_ora_results *Summarize ORA Results*

Description

Create a summary table of top enriched pathways across all clusters.

Usage

```
summarize_ora_results(
  ora_results,
  top_n = 10,
  p_threshold = 0.05,
  min_enrichment_ratio = 1.5
)
```

Arguments

ora_results	Output from perform_ora()
top_n	Number of top results per cluster to include
p_threshold	Adjusted p-value threshold for significance
min_enrichment_ratio	Minimum enrichment ratio threshold

Value

Data frame summarizing top enrichment results

Examples

```
## Not run:
# Summarize ORA results
summary_table <- summarize_ora_results(ora_results, top_n = 5)

## End(Not run)
```

wss_score	<i>Compute Within-Cluster Sum of Squares (WSS)</i>
-----------	--

Description

Compute Within-Cluster Sum of Squares (WSS)

Usage

```
wss_score(data, clusters)
```

Arguments

data	Numeric matrix or data frame of expression data (genes x samples)
clusters	Named vector of cluster assignments

Value

Total within-cluster sum of squares (numeric)

Examples

```
# Create sample gene expression data
set.seed(123)
data <- matrix(rnorm(200), nrow = 20, ncol = 10)
rownames(data) <- paste0("Gene", 1:20)
colnames(data) <- paste0("Sample", 1:10)

# Create cluster assignments
clusters <- c(rep(1, 8), rep(2, 7), rep(3, 5))
names(clusters) <- rownames(data)

# Compute WSS
wss_result <- wss_score(data, clusters)
message(paste("Total WSS: ", round(wss_result, 2)))
```

Index

* datasets

eset, [6](#)

cluster_expression, [2](#)

create_example_gene_sets, [3](#)

dbscan_cluster, [4](#)

detect_outliers, [5](#)

eset, [6](#)

extract_gene_names, [6](#)

filter_low_expression, [7](#)

gap_statistic, [8](#)

gap_statistic_parallel, [9](#)

get_cluster_results, [10](#)

hierarchical_cluster, [11](#)

kmeans_cluster, [12](#)

kmeans_parallel_k, [13](#)

load_gmt_gene_sets, [14](#)

load_online_gene_sets, [14](#)

normalize_data, [15](#)

ora_from_expression_set, [16](#)

perform_batch_correction, [17](#)

perform_normalization, [17](#)

perform_ora, [18](#)

perform_ora_parallel, [19](#)

plot_cluster_profiles, [20](#)

plot_dbscan_results, [20](#)

plot_dendrogram, [21](#)

plot_elbow, [22](#)

plot_heatmap, [22](#)

plot_pca, [23](#)

read_expression, [24](#)

run_pipeline, [25](#)

silhouette_score, [28](#)

summarize_ora_results, [29](#)

wss_score, [30](#)